

# OBJECT-ORIENTED PROGRAMMING

Assignment 1 – Object-Oriented Programming Fundamentals in Java

## FULL PROJECT / LEARNING REPORT

| Item         | Details                                  |
|--------------|------------------------------------------|
| Student Name | DELVIN RAJ A/L SUBRAMANIAM               |
| Student ID   | 202503010016                             |
| Class Code   | BIT1123                                  |
| Programme    | Bachelor of Software Engineering (BSSCE) |
| Course       | Object-Oriented Programming              |
| University   | City University Malaysia                 |
| Repository   | DELVIN-RAJ-202503010016-                 |
| Coverage     | Week 1 – Week 10                         |

*Prepared from the submitted GitHub repository and source files*

## Report Note

This report aims to document the learning progression of the student as reflected in the source code of the project repository. The repository contains folders for each of the tutorial lessons from Week 1 to Week 10, along with a README file and other supporting files. The report is based on the repository itself, its README file and an examination of the source files included within the repository.

Another important observation made during the review of this repository is included in the report: despite the fact that the README file shows that the repository was created using Java programming languages, the Week 1 folder contains a file named helloworld.py instead of a HelloWorld.java file. Also, the Java files that were created during Week 10 of the project include Git conflict markers (such as <<<<<<< HEAD and >>>>>>>) that were never resolved by the authors of the project. These observations are included within the report as a means of providing a detailed technical review of the repository.

# Table of Contents

1. Introduction
2. Student and Repository Overview
3. Objectives of the Assignment
4. Development Environment and Technologies
5. Repository Structure
6. Weekly Development and Learning Activities
  - 6.1 Week 1 – Programming Fundamentals
  - 6.2 Week 2 – Classes and Objects
  - 6.3 Weeks 3–4 – Inheritance, Encapsulation and Method Overriding
  - 6.4 Week 5 – Object-Oriented Programming Practice
  - 6.5 Week 6 – Employee and Lecturer Inheritance
  - 6.6 Week 7 – Abstraction and Polymorphism
  - 6.7 Weeks 8–9 – Programming Practice, Testing and Debugging
  - 6.8 Week 10 – Java Swing GUI Quiz Application
7. Object-Oriented Programming Concepts Demonstrated
8. Program Design and Code Quality Review
9. Testing and Debugging
10. Challenges Encountered and Solutions
11. Git and GitHub Usage
12. Learning Outcomes
13. Self-Reflection
14. Future Improvements
15. Conclusion
16. References
- Appendix A – Repository File Inventory
- Appendix B – Important Technical Findings

## 1. Introduction

Object-Oriented Programming (OOP) is a programming model that organizes software according to objects, classes, attributes and methods. The BIT1123 Object-Oriented Programming Assignment 1 repository documents the progress of the project from simple programming exercises to an object-oriented design with a graphical user interface.

The repository is organized into weekly folders from Week 1 to Week 10. These weekly folders allow for the observation of how the programming concepts became progressively more advanced. In the early weeks, the programming concepts were introduced to the students in relation to program structure and syntax. Later on, however, the concepts of classes, constructors, inheritance, abstraction, polymorphism, method overriding and event-driven GUI programming were covered in the lessons.

The main purpose of this report is to explain the work contained in the repository, to connect the source code to the relevant OOP principles, to evaluate the quality of the implementation and to reflect upon the skills that were developed throughout the learning process.

## 2. Student and Repository Overview

| Field                | Information                                                     |
|----------------------|-----------------------------------------------------------------|
| Student              | DELVIN RAJ A/L SUBRAMANIAM                                      |
| Student ID           | 202503010016                                                    |
| Class                | BIT1123                                                         |
| Programme            | Bachelor of Software Engineering (BSSCE)                        |
| Course               | Object-Oriented Programming                                     |
| Institution          | City University Malaysia                                        |
| Assignment           | Assignment 1 – Object-Oriented Programming Fundamentals in Java |
| Repository           | DELVIN-RAJ-202503010016-                                        |
| Repository status    | Public GitHub repository                                        |
| Main branch observed | main                                                            |
| Weekly coverage      | Week 1 to Week 10                                               |

The repository's README states that the repository contains exercises and projects completed in Java throughout the semester and that the main technologies and tools utilized in these projects are Java, Java OOP, Java Swing GUI, Git and GitHub

### 3. Objectives of the Assignment

- Get a solid grasp of Java's syntax and how programs are put together.
- Learn how to create and use classes and objects.
- Work with constructors, attributes, and methods.
- Use encapsulation to manage access to your class data.
- Understand inheritance so you can reuse code from parent classes.
- Practice method overriding to give child classes their own unique behaviors.
- Grasp abstraction using abstract classes and methods.
- See how polymorphism lets related classes handle common tasks in different ways.
- Build simple event-driven graphical user interfaces with Java Swing.
- Use Git and GitHub to manage, save, and document your coding projects.
- Sharpen your debugging, testing, and problem-solving abilities through consistent practice.

### 4. Development Environment and Technologies

| Technology / Tool | Purpose in the Repository                                                                       |
|-------------------|-------------------------------------------------------------------------------------------------|
| Java              | Primary programming language for the OOP tutorials and applications.                            |
| Java OOP          | Used for classes, objects, constructors, inheritance, abstraction, polymorphism and overriding. |
| Java Swing        | Used for the Week 10 graphical quiz application.                                                |

|                     |                                                                        |
|---------------------|------------------------------------------------------------------------|
| Git                 | Version-control tool used to track project changes.                    |
| GitHub              | Remote repository used to store, organize and document the assignment. |
| IDE / Java compiler | Used to compile and execute Java source files during development.      |

The README shows you how to use the command line with `javac` and `java`. For instance, to compile a Java main class, you'd type `javac Main.java`, and then to run it, `java Main`. If you're working on the Week 10 GUI project, the README says to compile `Questions.java` and `QuizBattleGUI.java` first, then run `QuizBattleGUT`.

## 5. Repository Structure

The repository is organized into weekly folders. This setup is handy because it lets us build and test each tutorial separately. Right at the top, you'll find a README.md file, along with these learning folders:

| Folder   | Observed Contents / Purpose                                                                       |
|----------|---------------------------------------------------------------------------------------------------|
| week_1   | helloworld.py; introductory programming work.                                                     |
| week_2   | Main.java and Student.java; classes and objects.                                                  |
| week_3_4 | Person.java, Student.java, Lecturer.java and Main.java; inheritance and overriding.               |
| week_5   | Main.java, Student.java and documentation.txt; OOP practice.                                      |
| week_6   | Employee.java, Lecturer.java and Main.java; inheritance and class relationships.                  |
| week_7   | Main.java, Microwave.java, WashingMachine.java and appliances.java; abstraction and polymorphism. |
| week_8_9 | Main.java and tasks.txt; programming exercises and practice.                                      |
| week_10  | Questions.java and QuizBattleGUI.java; Java Swing GUI quiz application.                           |

The repository contains 12 commits according to the GitHub repository page. The weekly folder structure and README provide a clear high-level record of the learning progression.

## 6. Weekly Development and Learning Activities

### 6.1 Week 1 – Programming Fundamentals

Week 1 is the starting point for our programming adventure, according to the README. It mentions basic Java exercises like "HelloWorld" and "StudentGrade," designed to teach us about syntax, how programs are put together, variables, printing things out, and simple logic.

But, the Week 1 folder in the repository actually has a file named `helloworld.py`. That's Python, not Java. If Week 1 is supposed to be in Java, this needs to be fixed. The programming ideas are fine for beginners, but the language and file type should match what the assignment says.

The goal here is to build a solid programming base before we get into classes and object-oriented design.

## 6.2 Week 2 – Classes and Objects

In Week 2, we'll clearly separate the class definition from the program that actually uses objects. The `Student` class will have attributes for name, age, and GPA. It will also include a constructor and methods called `displayInfo()`, `study()`, and `takeExam()`.

The constructor will take `studentName`, `studentAge`, and `studentGpa` as input, then assign these to the object's attributes. This shows how we can give an object useful starting values when we create it.

The `displayInfo()` method will print out the object's current details. Meanwhile, `study()` and `takeExam()` will represent actions a `Student` object can perform. This directly illustrates the core OOP concept that objects bundle together data and the actions that go with it.

Example design extracted from the repository:

```
public class Student {
    String name;
    int age;
    double gpa;

    public Student(String studentName, int studentAge, double studentGpa) {
        name = studentName;
        age = studentAge;
        gpa = studentGpa;
    }
}
```

## 6.3 Weeks 3–4 – Inheritance, Encapsulation and Method Overriding

Weeks 3 and 4 shift from looking at single objects to how they relate to each other. The repository has three classes: Person, Student, and Lecturer. The Person class holds a person's name and ID in private fields, offering public getter methods to access them. This shows encapsulation, as you can't directly mess with the fields.

Both Student and Lecturer inherit from Person. Each one calls the parent constructor using `super(name, id)`. They also both override the `introduce()` method. This means the same method name can give different results depending on what kind of object it's actually working with.

This is a key step beyond basic object-oriented programming, moving into inheritance and how things behave when the program is running. The parent class lays out the shared details and actions, while the child classes put their own spin on how those actions are carried out.

#### 6.4 Week 5 – Object-Oriented Programming Practice

Week 5 brings Main.java, Student.java, and documentation.txt. The goal here is to really nail down the class-and-object design we've been working on. Keeping the Student class separate from Main helps organize the program's pieces. It also shows that you can create a class on its own, apart from the code that actually makes and uses its objects.

The documentation file is handy too, offering extra info right alongside the source code. This is how good software development works: you need to write enough documentation so someone else can figure out what your code is doing.

#### 6.5 Week 6 – Employee and Lecturer Inheritance

In week 6, we see another example of inheritance with the Employee and Lecturer classes. The Employee class has protected `'id'` and `'name'` attributes, along with a constructor and a `'displayInfo()'` method. The Lecturer class builds on Employee, extending it and adding its own `'subject'` and `'department'` fields.

The Lecturer constructor uses `'super(id, name)'` to call the Employee constructor, effectively reusing the initialization code. Lecturer also adds its own methods, `'displaySubject()'` and `'displayDepartment()'`, to show its specific details.

This setup really highlights code reuse. The Lecturer class doesn't have to redefine `'id'`, `'name'`, or the `'displayInfo()'` functionality. Instead, it simply inherits these from the Employee class.

#### 6.6 Week 7 – Abstraction and Polymorphism

In week 7, you'll find an abstract Appliance class, along with two subclasses: Microwave and WashingMachine. The Appliance class has a protected brand attribute, a constructor, and methods for displaying the brand, turning the appliance on, and turning it off. It also includes an abstract `operate()` method.

Because `operate()` is abstract, each subclass must define its own version. The Microwave class will implement it to display 'Heating food...', and the WashingMachine class will use 'Washing clothes...'. This clearly shows abstraction and polymorphism in action.

This design works well. It keeps the shared characteristics of appliances in the main Appliance class, while letting each specific appliance type handle its own unique operation in its own child class.



## 6.7 Weeks 8–9 – Programming Practice, Testing and Debugging

Weeks 8 and 9 feature `Main.java` and `tasks.txt`. The README explains these weeks are for extra Java practice, meant to solidify earlier ideas and give you hands-on experience with writing, testing, and debugging code.

These weeks matter because you get better at programming by doing it over and over and fixing your mistakes. Even if these exercises are smaller than the Week 10 GUI project, they're still great chances to work on syntax, logic, how to structure classes, and how to fix errors.

## 6.8 Week 10 – Java Swing GUI Quiz Application

Week 10 is where the rubber meets the road, application-wise. You'll find `Questions.java` and `QuizBattleGUI.java` here. The idea is to build a simple programming quiz interface using Java Swing.

`QuizBattleGUI` is set up as a `JFrame` and also implements `ActionListener`. It's responsible for creating labels and buttons, arranging them in a window, setting up action listeners so it knows when a button is clicked, and then reacting to those clicks. The `Questions` class, on the other hand, holds a question, two answer choices, and the correct answer. It also has a handy `isCorrect()` method.

The user experience is meant to be straightforward: the app shows a programming question, offers two choices, and then updates a result label based on whether the user picked the right answer.

There's a pretty big technical snag in the current Week 10 files, though. Both `Questions.java` and `QuizBattleGUI.java` have leftover Git merge-conflict markers. This means two different versions of the files were merged together, but the merge wasn't finished properly. Because of this, the files aren't valid Java code right now and need to be fixed before they're submitted.

This conflict also highlights that the repository has two different ways of handling the constructor. One version sets up a default question when you create a `Questions` object, while another uses a constructor that takes arguments. For a quiz app that you might want to expand later, using the parameterized constructor is the better approach. It makes it easier to create different question objects without having to hard-code all the details directly into the class.

## 7. Object-Oriented Programming Concepts Demonstrated

| Concept    | Evidence in Repository                          | Explanation                                                    |
|------------|-------------------------------------------------|----------------------------------------------------------------|
| Class      | Student, Person, Employee, Appliance, Questions | Classes define the structure and behaviour of related objects. |
| Object     | Main classes create/use class instances         | Objects represent concrete instances of classes.               |
| Attributes | name, age, gpa, id, subject,                    | Attributes store object state.                                 |

|                      |                                                      |                                                                    |
|----------------------|------------------------------------------------------|--------------------------------------------------------------------|
|                      | department, brand                                    |                                                                    |
| Methods              | displayInfo(), study(), operate(),<br>isCorrect()    | Methods represent object behaviour.                                |
| Constructor          | Student(), Person(), Employee(),<br>Lecturer()       | Constructors initialize objects.                                   |
| Encapsulation        | private fields in Person; getters                    | Access to internal data is controlled.                             |
| Inheritance          | Student extends Person; Lecturer<br>extends Employee | Child classes reuse parent features.                               |
| Method overriding    | introduce() and operate()                            | Child classes provide specialized implementations.                 |
| Abstraction          | abstract class Appliance; abstract<br>operate()      | Common structure is defined while specific behaviour is deferred.  |
| Polymorphism         | Microwave and WashingMachine<br>operate()            | The same operation can behave differently in different subclasses. |
| GUI / event handling | QuizBattleGUI, ActionListener                        | User actions trigger application behaviour.                        |

## 8. Program Design and Code Quality Review

### 8.1 Strengths

- The learning path is clear because the repository is organized into weekly folders.
- Small, easy-to-grasp examples show how several OOP concepts work.
- In many weeks, class definitions are kept separate from the main or test classes.
- Objects are initialized using constructors, not just default values.
- Inheritance is shown multiple times, helping to really drive the concept home.
- Abstraction and polymorphism are demonstrated in Week 7, using an abstract parent class and overridden methods.
- Week 10 marks a shift from programming in the console to building event-driven GUIs.
- You'll find setup instructions and an overview of the learning journey in the README.
- GitHub serves as a single place for both the source code and its documentation.

### 8.2 Areas for Improvement

- Week 1's Java source code should be included if the assignment is purely Java-based. The 'helloworld.py' file needs to be either removed or its presence clearly explained.
- Week 10 has unresolved Git merge-conflict markers that need to be fixed before you submit it.
- The class name 'appliances.java' doesn't match the 'Appliance' class name. While Java technically allows a non-public class in a file with a different name, it's better practice to fix this for a more professional look.

- Some of the earlier fields are set to package-private access. Using private fields with getters and setters more consistently would make the encapsulation stronger.
- Make sure variable and class names consistently follow standard Java naming conventions.
- The Week 10 quiz could be updated to handle more than one question, rather than having just one question hard-coded.
- Consider using layout managers for the GUI instead of a null layout. This will make it easier to adapt and maintain the code later on.
- Input handling could be made more reliable, especially if you plan to expand the quiz in the future.
- Adding more comments and documentation would help explain any design choices that aren't immediately obvious.

## 9. Testing and Debugging

Testing in this project is primarily based on running the individual weekly programs and observing their output. This is suitable for beginner OOP exercises because the expected results are simple and visible.

| Testing Area           | Expected Result                                                                                             |
|------------------------|-------------------------------------------------------------------------------------------------------------|
| Week 2 Student         | Student details are displayed and study/exam messages are printed.                                          |
| Weeks 3–4 inheritance  | Student and Lecturer objects use inherited identity information and their overridden introduce() behaviour. |
| Week 6 Lecturer        | Employee information plus lecturer subject and department can be displayed.                                 |
| Week 7 Microwave       | The appliance can display its brand, power state and microwave-specific operation.                          |
| Week 7 WashingMachine  | The appliance can display its brand, power state and washing-machine-specific operation.                    |
| Week 10 correct answer | Result label displays a correct-answer message.                                                             |
| Week 10 wrong answer   | Result label displays a try-again message.                                                                  |

The key debugging discovery is the merge conflict from Week 10. Those conflict markers show the source code needs cleaning up in version control before it can be considered a finished Java program. To fix it properly, pick one constructor's code, get rid of all the conflict markers, compile the two files, and then test both answer buttons.

## 10. Challenges Encountered and Solutions

| Challenge                       | Impact                                                  | Solution / Learning                                                               |
|---------------------------------|---------------------------------------------------------|-----------------------------------------------------------------------------------|
| Understanding OOP relationships | It can be difficult to understand how classes interact. | Use small parent-child examples and trace which fields and methods are inherited. |

|                              |                                                             |                                                                                                           |
|------------------------------|-------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------|
| Understanding constructors   | Incorrect initialization can cause unexpected object state. | Pass values through constructors and use <code>super()</code> when initializing inherited data.           |
| Understanding overriding     | A child method can replace parent behaviour.                | Compare the parent method with the <code>@Override</code> implementation in the child class.              |
| Abstraction and polymorphism | The concepts can appear theoretical at first.               | The Appliance/Microwave/WashingMachine example connects the concepts to familiar objects.                 |
| GUI event handling           | Button clicks require event-driven programming.             | Implement <code>ActionListener</code> and inspect the event source to determine which button was clicked. |
| Git merge conflict           | Conflicting versions were left inside Week 10 files.        | Resolve the conflict manually, remove markers, compile, run and commit the corrected version.             |

## 11. Git and GitHub Usage

The assignment uses GitHub as its remote repository. This repository is public and set up with a main branch and folders for each week. You'll find a README file there that describes the project and explains how to run the programs.

Using GitHub is a good practice for software development. It keeps a record of all the changes made and offers a clear place to store source code and documentation. The repository shows 12 commits, meaning the assignment was built through several distinct steps.

However, the merge conflict that happened in Week 10 highlights the need for careful work with version control. A merge isn't truly done until all conflict markers are gone, the program has been tested, and the updated files have been committed.

## 12. Learning Outcomes

- I know how to make objects and set them up using constructors.
- I get why inheritance helps us avoid writing the same code over and over.
- I understand how a child class can change a method it gets from its parent.
- I can spot encapsulation when I see private fields and public methods working together.
- I understand what abstract classes and abstract methods are for.
- I can explain polymorphism by showing how different subclasses can do the same thing in their own way.
- I've gotten my first taste of Java Swing and handling events.
- I'm better at keeping my source code organized in folders and separate classes.

- I've used GitHub to store code and write documentation.
- I've learned that testing and debugging are just part of programming, not something you do at the very end.

## 13. Self-Reflection

When I first started learning to program, it was all about the basics: syntax, variables, and just getting simple output. But as the tutorials went on, the focus shifted. We moved from writing individual lines of code to designing classes and figuring out how they'd relate to each other. This really brought more structure to programming, letting us group data and actions into logical objects.

Learning about inheritance was a huge takeaway. The examples with Person, Student, and Lecturer, and then Employee and Lecturer, really drove home how a general class can hold shared information, while a more specific class can add or tweak how things work. It made the idea of reusing code feel much more concrete and easier to grasp.

The Appliance example also boosted my understanding of abstraction and polymorphism. Instead of designing each appliance from scratch, we could put common features and actions into a parent class. Then, each specific appliance could handle its own version of the `operate()` function.

The GUI project in Week 10 was a big leap. It introduced a whole new way of programming. Console programs usually just run through a list of instructions, but a GUI program sits and waits for the user to do something, then reacts to those actions. Learning about `ActionListener` was key to connecting object-oriented programming with how real applications actually interact with users.

Looking back at the repository, I see that software development really involves both version control and managing quality. That merge conflict we couldn't sort out in Week 10 taught me a lot. Just because code seems right in my head doesn't mean it's ready to go. It actually needs to compile, pass tests, be neatly versioned, and have good documentation.

All in all, this assignment has made me feel much more confident with Java and object-oriented programming. I'm now on much firmer ground for building bigger applications where lots of different classes need to work together smoothly.

## 14. Future Improvements

- Replace the Week 1 Python file with the required Java implementation, if Java is required by the module.
- Resolve and commit the Week 10 merge conflict.
- Rename `appliances.java` to `Appliance.java` for consistent Java naming.
- Use private fields consistently and expose data through suitable getters/setters where needed.
- Add validation for values such as GPA or empty strings where appropriate.

- Expand the quiz to support multiple Questions objects stored in an array or collection.
- Add a score counter and a question counter to the GUI.
- Use a layout manager such as BorderLayout, GridLayout or BoxLayout instead of fixed null-layout coordinates.
- Add automated or repeatable test cases for important methods.
- Improve README documentation with screenshots, expected outputs and exact Java version requirements.
- Continue practicing interfaces, collections, exception handling, file I/O and more advanced GUI design.

## 15. Conclusion

The BIT1123 Assignment 1 repository shows a solid learning curve in basic Object-Oriented Programming. It starts with introductory programming and then moves into classes, objects, constructors, encapsulation, inheritance, overriding, abstraction, polymorphism, and GUI event handling.

What really stands out in the repository is how the examples build on each other. Using exercises based on Students, People, Lecturers, Employees, and Appliances helps make abstract OOP ideas feel more concrete and easier to relate to the actual code. The Week 10 quiz project then brings together class design with Java Swing and user interaction.

Overall, the repository is pretty well organized, but it needs a few technical fixes to be truly polished. Specifically, the language/file mix-up in Week 1 and the unresolved Git merge conflict from Week 10 should be sorted out. Fixing these, making the naming more consistent, and adding more to the GUI would really strengthen the submission.

To wrap up, this assignment has laid a practical groundwork in Java Object-Oriented Programming and software development processes. The skills learned here can serve as a solid base for building larger Java applications and for future software engineering projects.

## 16. References

1. DELVIN RAJ A/L SUBRAMANIAM. BIT1123 Object-Oriented Programming – Assignment 1 repository. GitHub. Repository: <https://github.com/dcruzz1213-cmd/DELVIN-RAJ-202503010016->
2. Repository README.md – student information, assignment description, repository structure, weekly tutorial summaries, technologies, running instructions and reflection summary.
3. Repository source files – Week 2 through Week 10 Java source files reviewed for class design, inheritance, abstraction, polymorphism and GUI implementation.

## Appendix A – Repository File Inventory

| Week      | Files observed                                                  | Main focus                                         |
|-----------|-----------------------------------------------------------------|----------------------------------------------------|
| Week 1    | helloworld.py                                                   | Introductory programming; language mismatch noted. |
| Week 2    | Main.java; Student.java                                         | Classes, objects, constructor, methods.            |
| Weeks 3–4 | Person.java; Student.java; Lecturer.java; Main.java             | Inheritance, encapsulation, overriding.            |
| Week 5    | Main.java; Student.java; documentation.txt                      | OOP practice and documentation.                    |
| Week 6    | Employee.java; Lecturer.java; Main.java                         | Inheritance and class relationships.               |
| Week 7    | Main.java; Microwave.java; WashingMachine.java; appliances.java | Abstraction and polymorphism.                      |
| Weeks 8–9 | Main.java; tasks.txt                                            | Programming practice and debugging.                |
| Week 10   | Questions.java; QuizBattleGUI.java                              | Java Swing GUI and event handling.                 |

## Appendix B – Important Technical Findings

Finding 1 – Week 1 file type: The current repository folder contains helloworld.py. The README describes Week 1 as Java basics. If the module requires Java, the file should be replaced with a Java source file such as HelloWorld.java.

Finding 2 – Week 7 filename: The source file is named appliances.java while the class declaration is Appliance. For professional Java project organization, the file should be renamed to Appliance.java.

Finding 3 – Week 10 merge conflict: Questions.java and QuizBattleGUI.java contain Git conflict markers such as <<<<<<< HEAD, ===== and >>>>>>. These markers are not valid Java syntax and must be removed by resolving the conflicting versions.

Finding 4 – Week 10 design choice: The parameterized Questions constructor is more flexible for future quiz expansion because each question can be created with its own text, options and answer.

Finding 5 – Testing: After resolving the Week 10 conflict, compile both Java files together and run the GUI. Verify the correct and incorrect answer paths before committing the corrected version.